

Patient Management System

REST API Documentation

Framework:	FastAPI (Python)
Validation:	Pydantic v2
Storage:	JSON File (patients.json)
Server:	Uvicorn (ASGI)
Auth:	None (Development API)

1. Overview

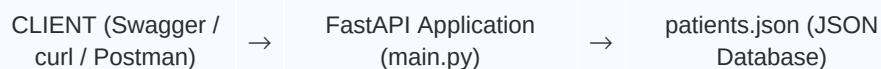
The Patient Management System API is a RESTful backend service built with FastAPI. It enables complete CRUD (Create, Read, Update, Delete) operations on patient records stored in a local JSON file. The system auto-calculates BMI and generates a health verdict for every patient.

Key Features

- Full CRUD operations: Create, View, Update, Delete patients
- Auto-computed BMI from height and weight
- Auto-generated health verdict (Underweight / Normal / Obese)
- Query-based sorting by height, weight, or BMI
- Built-in validation with Pydantic v2 (type-safe, annotated)
- Interactive Swagger UI at /docs
- Proper HTTP status codes and error handling via HTTPException

2. System Architecture

The API follows a simple 3-layer architecture — Client, Application Logic, and Data Store.



The client (browser, Postman, curl) sends HTTP requests to the FastAPI application. FastAPI routes the request to the correct handler, which loads data from patients.json, applies the business logic, and returns a JSON response.

Project File Structure

The project consists of three core files that work together to deliver a lightweight, file-based patient records API.

File	Purpose
main.py	Core application — models, routes, logic
patients.json	JSON flat-file database storing all patient records
requirements.txt	Python package dependencies

3. Request Lifecycle

Every HTTP request goes through the following stages inside the application:

1	Client sends HTTP Request
2	FastAPI Router matches endpoint
3	Pydantic validates request body
4	load_data() reads patients.json
5	Business logic executes
6	save_data() writes to patients.json
7	JSONResponse returned to client

Steps 4 and 6 (JSON read/write) only apply to routes that interact with the data store.

4. Data Models

4.1 Patient Model (Input + Computed)

The Patient model is defined using Pydantic v2 and handles both incoming request validation and auto-computation of derived fields.

Field	Type	Status	Description
id	str	Required	Unique patient ID e.g. P001
name	str	Required	Full name of the patient
city	str	Required	City of residence
age	int	Required	Age: must be > 0 and < 120
gender	Enum	Required	male female others
height	float	Required	Height in metres (must be > 0)
weight	float	Required	Weight in kg (must be > 0)
bmi	float	Computed	weight / height ² (auto-calculated)
verdict	str	Computed	Underweight / Normal / Obese

4.2 PatientUpdate Model (PATCH-style PUT)

All fields in PatientUpdate are optional. Only the fields provided in the request body are updated — the rest remain unchanged. This is enabled by `model_dump(exclude_unset=True)`.

Field	Type	Notes
name	Optional	Defaults to None — only updated if provided in request
city	Optional	Defaults to None — only updated if provided in request
age	Optional	Defaults to None — only updated if provided in request
gender	Optional	Defaults to None — only updated if provided in request
height	Optional	Defaults to None — only updated if provided in request
weight	Optional	Defaults to None — only updated if provided in request

4.3 BMI Verdict Logic

BMI is automatically calculated from the patient's height and weight using the formula: $\text{BMI} = \text{weight} / \text{height}^2$. The verdict is then assigned based on the following ranges:

BMI Range	Verdict
< 18.5	Underweight
18.5 – 24.9	Normal
25.0 – 29.9	Overweight ✓ Fixed
>= 30	Obese

5. API Endpoints

The following table summarises all available REST endpoints, their HTTP methods, expected status codes, and descriptions.

Method	Endpoint	Status	Description
GET	/	200	Welcome message
GET	/about	200	About API description
GET	/view	200	All patient records
GET	/patient/{patient_id}	200 / 404	Single patient by ID
GET	/sort	200 / 400	Sorted patient list
POST	/create	201 / 400	Create new patient
PUT	/edit/{patient_id}	200 / 404	Update patient fields
DELETE	/delete/{patient_id}	200 / 404	Remove patient record

6. Endpoint Details

GET / and GET /about

Simple informational endpoints. No parameters, no DB interaction.

Response: { "message": "Patient Management System API" }

GET /view

Returns the full contents of patients.json as a dictionary, keyed by patient ID.

Response: { "P001": { "name": "...", "bmi": 22.1, ... }, ... }

GET /patient/{patient_id}

Returns a single patient record by ID. Raises HTTP 404 if the ID does not exist.

- Path Parameter: patient_id (str) — example: P001
- Success: 200 + patient object
- Error: 404 + { "detail": "Patient not found" }

GET /sort

Returns all patients sorted by a chosen field in ascending or descending order.

Param	Values	Description
sort_by	height, weight, bmi	Field to sort on
order	asc, desc	Sort direction (default: asc)

Example: GET /sort?sort_by=bmi&order=desc

POST /create

Creates a new patient record. The request body must include all required fields. BMI and verdict are auto-calculated by Pydantic. Returns HTTP 400 if the patient ID already exists.

Body: { "id": "P001", "name": "Jane", "city": "Mumbai", "age": 25, "gender": "female", "height": 1.60, "weight": 55.0 }

- Success: 201 + { "message": "patient created successfully" }
- Error: 400 + { "detail": "Patient already exists" }

PUT /edit/{patient_id}

Updates one or more fields of an existing patient. Only fields included in the request body are modified. After update, BMI and verdict are recalculated automatically by converting the record back into a Pydantic object.

Body (partial): { "weight": 68.5 }

- Success: 200 + { "message": "patient updated" }
- Error: 404 + { "detail": "Patient not found" }

DELETE /delete/{patient_id}

Permanently removes a patient record from the JSON database.

- Success: 200 + { "message": "patient deleted" }
- Error: 404 + { "detail": "Patient not found" }

7. Helper Functions

`load_data()`

Opens `patients.json` in read mode and deserializes the JSON content into a Python dictionary. Called at the start of every route handler.

`save_data(data)`

Accepts a Python dictionary and serializes it back to `patients.json`. Called after every write operation (create, update, delete).

8. Error Handling

The API uses `HTTPException` for all error scenarios. The table below lists the standard HTTP error codes and the conditions that trigger them.

HTTP Code	Scenario	Example Detail Message
400	Duplicate patient ID	Patient already exists
400	Invalid sort_by field	Invalid field select from ['height','weight','bmi']
400	Invalid sort order	Invalid order select between asc and desc
404	Patient not found	Patient not found
422	Pydantic validation fail	Automatic — generated by FastAPI

9. Setup & Running

Installation Steps

Follow the steps below to get the API running locally on your machine:

1	Clone repo and cd into project folder
2	Create virtual environment: <code>python -m venv venv</code>
3	Activate venv and run: <code>pip install fastapi uvicorn pydantic</code>
4	Create empty database: <code>echo "{}" > patients.json</code>
5	Start server: <code>uvicorn main:app --reload</code>
6	Visit http://127.0.0.1:8000/docs for Swagger UI